

COMPLETE GUIDE

RAG Patterns

From Basic RAG to Agentic, Speculative & Graph RAG

Every pattern explained

problem → diagram → when to use

AI Coach John

PROITBRIDGE
Strive For Better Future

Follow

Repost

⚠ CONTEXT

Why Patterns Matter

Basic RAG works for demos. Real production apps break it in dozens of ways. Patterns are proven solutions to those specific failure modes.

```
Basic RAG fails when:
─────────────────────
✗ Retrieved docs are irrelevant
✗ Query is ambiguous or poorly worded
✗ Answer needs info from multiple docs
✗ Documents contradict each other
✗ Question needs multi-step reasoning
✗ Knowledge graph relationships matter
✗ Real-time web info is needed
```

```
Pattern Map by Complexity:
──────────────────────────
Beginner  → Naive RAG, HyDE, Rewrite-Retrieve
Intermediate → CRAG, Self-RAG, Adaptive RAG,
        Speculative RAG, FLARE
Advanced  → Modular RAG, Agentic RAG,
       GraphRAG, Multi-Modal RAG
```


AI Coach John
PROITBRIDGE
Strive For Better Future
Follow
Repost

PATTERN 1

Naive RAG (Baseline)

Problem: Getting started the foundation every other pattern builds on.

```
User Query
  |
  ▼
[Vector Search]
  |
  ▼
Top-K Chunks Retrieved
  |
  ▼
[LLM Prompt]
Query + Chunks → Answer
  |
  ▼
Response to User
```

HOW IT WORKS

- Convert query to embedding
- Find top-k similar chunks in vector DB
- Stuff chunks into LLM prompt
- LLM generates answer from context

- ✓ Simple Q&A over clean documents
- ✓ Prototype / proof of concept
- ✗ Avoid when queries are ambiguous
- ✗ Avoid when docs are noisy or outdated


AI Coach John
PROITBRIDGE Strive For Better Future
Follow
Repost

PATTERN 2

HyDE (Hypothetical Doc Embeddings)

Problem: Short user queries embed poorly they don't match the style of documents in the index, causing weak retrieval.

```
User Query: "HNSW?"
  |
  ▼
[LLM] Generate Hypothetical Answer
  |
  ▼
[Embed the Hypothetical Answer]
  | (not the original query)
  ▼
[Vector Search with rich embedding]
  |
  ▼
Real Chunks Retrieved (much better match)
```

HOW IT WORKS

- LLM first imagines what a good answer would look like
- That imagined answer is embedded
- Rich embedding finds much better matches in vector DB

- ✓ Queries are very short (1-3 words)
- ✓ Users ask in conversational style
- ✗ Avoid when LLM hallucination risk is high
- ✗ Avoid when latency matters 2 LLM calls


AI Coach John
PROITBRIDGE Strive For Better Future
Follow
Repost

PATTERN 3

Query Rewriting

Problem: Users write vague, poorly structured, or ambiguous queries that don't retrieve well as-is.

```
User Query: "that thing about splitting docs"
  |
  ▼
[LLM] Rewrite Query
  ├── "What is text chunking in RAG?"
  ├── "How to split documents for LLM?"
  └── "Document splitting strategies"
  |
  ▼
[Vector Search all 3 rewrites]
  |
  ▼
[Deduplicate Results] → [LLM] Final Answer
```

HOW IT WORKS

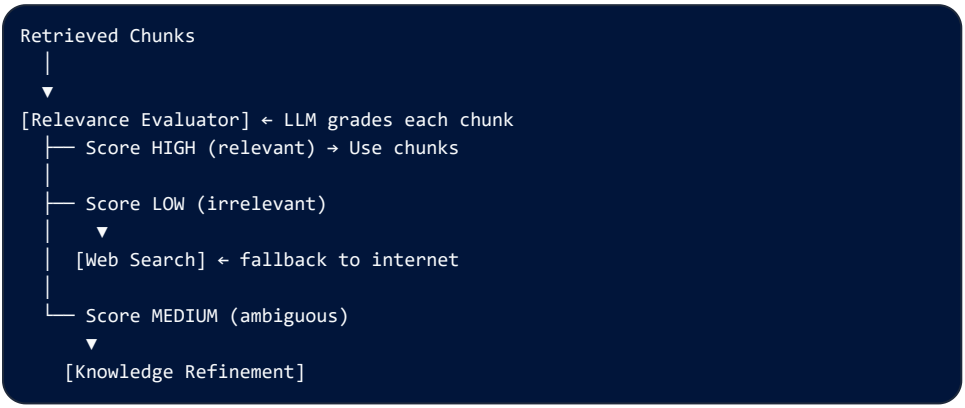
- LLM analyzes original query for ambiguity
- Generates cleaner / broader / rephrased versions
- Each version retrieves independently
- Results merged, deduplicated, sent to LLM

- ✓ Chatbot users type casually
- ✓ Domain with lots of jargon/abbreviations
- ✗ Avoid when queries are already precise
- ✗ Adds latency extra LLM call per query

PATTERN 4

CRAG (Corrective RAG)

Problem: Retrieved documents are sometimes irrelevant or wrong Naive RAG passes them to the LLM anyway.



HOW IT WORKS

- Grades each chunk: relevant / irrelevant / ambiguous
- Irrelevant triggers web search as fallback
- Ambiguous strips bad parts, keeps good parts
- Only clean context reaches LLM

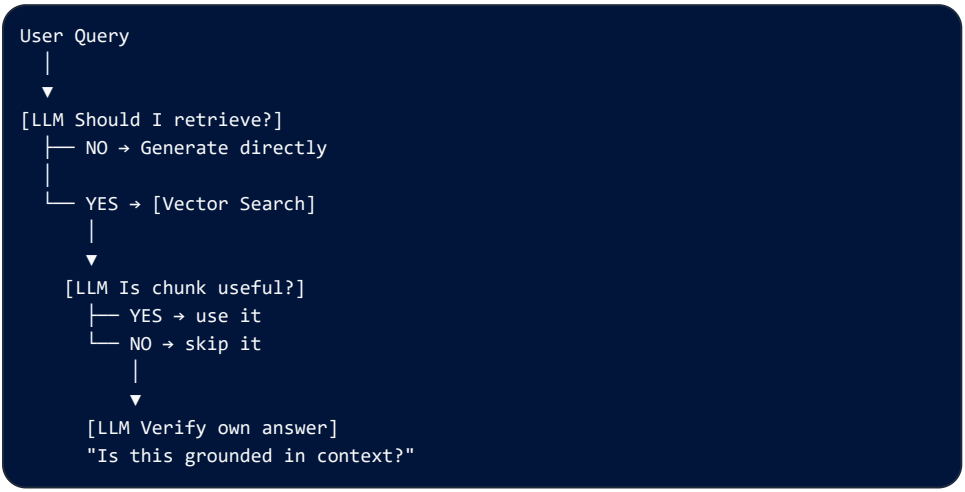
- ✓ Knowledge base may be incomplete
- ✓ Accuracy is critical (legal, medical)
- ✗ Slower evaluation adds latency
- ✗ Avoid when web search is not permitted


AI Coach John
PROITBRIDGE Strive For Better Future
Follow
Repost

PATTERN 5

Self-RAG

Problem: LLM blindly generates from retrieved context even when retrieval wasn't needed.



HOW IT WORKS

- LLM decides IF retrieval is needed
- Generates answer only from relevant chunks
- Critically evaluates its own output for faithfulness
- Regenerates if answer isn't grounded

- ✓ Mixed queries some need docs, some don't
- ✓ Faithfulness to source is non-negotiable
- ✗ Slow multiple LLM calls per query
- ✗ Needs fine-tuned model for best results


AI Coach John PROITBRIDGE Strive For Better Future Follow Repost

PATTERN 6

Adaptive RAG

Problem: One retrieval strategy doesn't fit all queries simple questions waste compute on complex pipelines.

```
User Query
  |
  ▼
[Query Complexity Classifier]
  |
  ├── SIMPLE (factual, direct)
  |   → [Single Vector Search]
  |
  ├── MEDIUM (needs context)
  |   → [Standard RAG Pipeline]
  |
  └── COMPLEX (multi-hop, reasoning)
      → [Agentic Pipeline]
        Multiple retrievals, Tool calls
```

HOW IT WORKS

- Classifier scores query complexity
- Routes to appropriate pipeline tier
- Simple queries skip expensive steps
- Complex queries get full agentic treatment

- ✓ Mixed workload varied query types
- ✓ Cost & latency optimization needed
- ✗ Classifier itself adds a small latency
- ✗ Needs tuning misclassification hurts quality



AI Coach John

PROITBRIDGE
Strive For Better Future

Follow

Repost

PATTERN 7

FLARE

Problem: Long-form generation loses accuracy midway LLM starts hallucinating when it moves beyond initial context.

```
[LLM starts generating...]
  |
  ▼
[Confidence Check word by word]
  | "...the dimensions are [UNSURE]"
  ▼
[Trigger Retrieval mid-generation]
  | Query: "embedding dimensions"
  ▼
[Retrieve fresh context]
  |
  ▼
[Resume generation with new context]
```

HOW IT WORKS

- LLM generates token by token
- Monitors its own confidence at each step
- When confidence drops, pauses and retrieves
- Resumes generation with new grounding

- ✓ Long-form generation (reports, summaries)
- ✓ Factual accuracy throughout is critical
- ✗ Short Q&A overkill, adds latency
- ✗ Needs LLM with reliable confidence scores



AI Coach John

PROITBRIDGE
Strive For Better Future

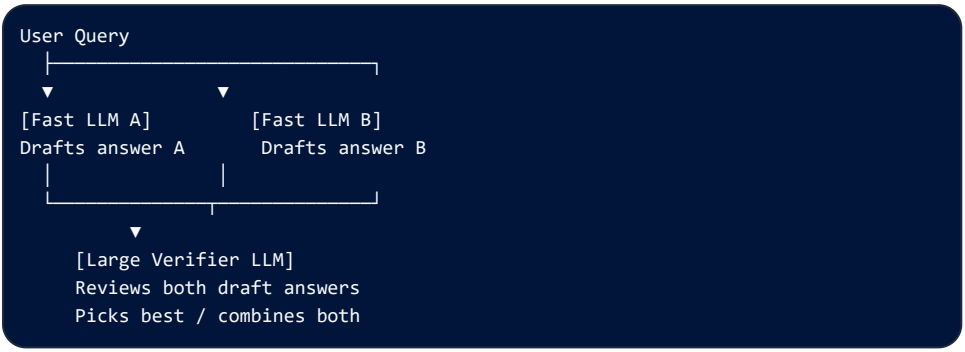
Follow

Repost

PATTERN 8

Speculative RAG

Problem: Single-LLM RAG is slow and bottlenecked. Accuracy and speed trade off against each other.



HOW IT WORKS

- Query and chunks split into subsets
- Multiple small specialist LLMs draft answers in parallel
- One large LLM acts as verifier/judge
- Parallel drafting = speed; large verifier = accuracy

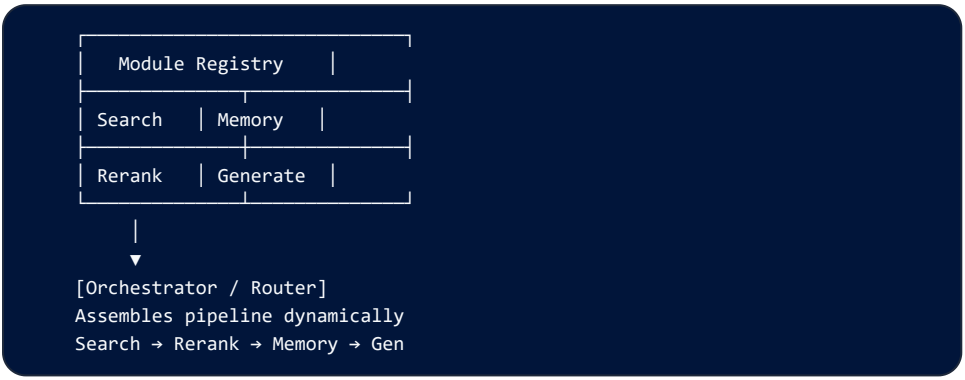
- ✓ Large document sets too big for one prompt
- ✓ Latency matters parallel drafts are faster
- ✗ Budget constrained uses multiple LLM calls
- ✗ Small knowledge bases unnecessary overhead


AI Coach John
PROITBRIDGE
Strive For Better Future
Follow
Repost

PATTERN 9

Modular RAG

Problem: Monolithic RAG pipelines are rigid you can't swap, skip, or reorder components easily.



HOW IT WORKS

- Every RAG component is a swappable module
- Orchestrator chains modules at runtime
- Add new modules without touching existing ones
- Different query types get different module chains

- ✓ Large teams working on same pipeline
- ✓ Long-term production system that will evolve
- ✗ Simple apps too much architecture overhead
- ✗ Solo projects YAGNI


AI Coach John
PROITBRIDGE Strive For Better Future
Follow
Repost

PATTERN 10

Agentic RAG

Problem: Complex multi-step questions can't be answered with a single retrieval.

```
User: "Compare chunking strategies"
  |
  ▼
[Agent Plans steps]
  ├── Step 1: retrieve "strategies"
  ├── Step 2: retrieve "tradeoffs"
  ├── Step 3: [Calculator Tool] estimate
  └── Step 4: [LLM Synthesize]
  |
  ▼
Full multi-part answer
```

HOW IT WORKS

- LLM agent receives query and available tools
- Plans a sequence of steps to answer fully
- Each step's output feeds the next
- Can loop, backtrack, and retry failed steps

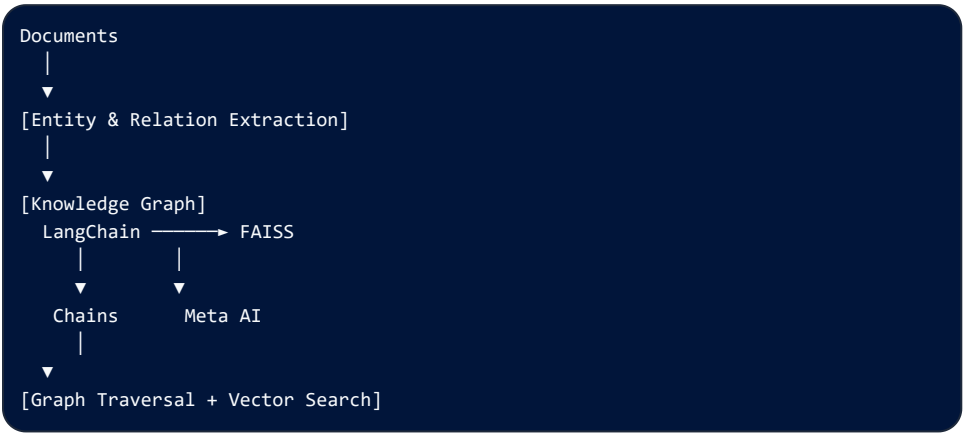
- ✓ Multi-hop questions (A depends on B)
- ✓ Mix of retrieval + computation + web
- ✗ Latency sensitive many sequential calls
- ✗ Hard to debug and test


AI Coach John
PROITBRIDGE Strive For Better Future
Follow
Repost

PATTERN 11

GraphRAG

Problem: Flat vector search misses relationships between entities how concepts connect across documents.



HOW IT WORKS

- Extract entities and relations from documents
- Build knowledge graph (nodes & edges)
- At query time, combine vector + graph traversal
- Retrieves chunks AND connected entities

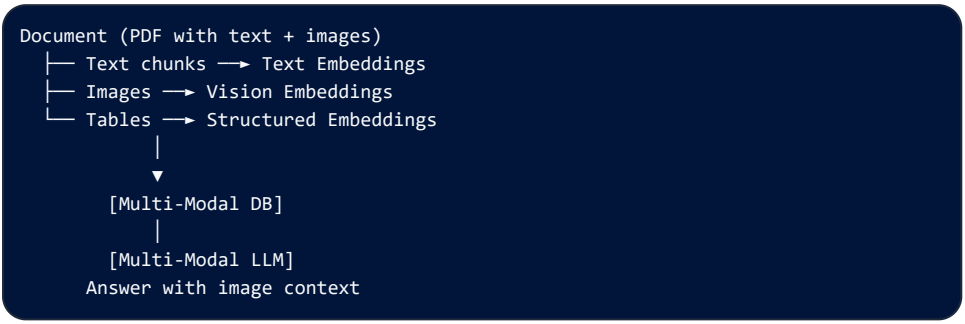
- ✓ Queries about relationships between entities
- ✓ Knowledge base has interconnected concepts
- ✗ Graph construction is expensive upfront
- ✗ Maintenance heavy graph needs updates


AI Coach John
PROITBRIDGE Strive For Better Future
Follow
Repost

PATTERN 12

Multi-Modal RAG

Problem: Real documents contain images, charts, tables text-only RAG ignores up to 50% of the info.



HOW IT WORKS

- Docs parsed into text, images, tables
- Each modality embedded appropriately (CLIP etc)
- Query searches across all modalities
- Multi-modal LLM reasons over combined context

- ✓ PDFs with diagrams, charts, screenshots
- ✓ Medical imaging + clinical notes
- ✗ Text-only documents no benefit
- ✗ Image embedding adds significant latency


AI Coach John
PROITBRIDGE Strive For Better Future
Follow
Repost

PATTERN 13

Conversational RAG

Problem: Each query is treated as independent chatbot forgets what was said 2 messages ago.

```
Turn 1: User: "What is RAG?"
  |
[Memory Store saves Q&A]

Turn 2: User: "How does it work?"
  |
  ▼
[Memory fetch last N turns]
  |
[Query Rewriter]
"What is RAG?" + "How does it work?"
→ "How does RAG work?"
  |
[RAG Retrieve + Generate]
```

HOW IT WORKS

- Maintains conversation history in memory store
- Query rewriter adds previous context to query
- Rewritten query retrieves better documents
- Handles pronouns, follow-ups, topic switches

- ✓ Chatbot / assistant interface
- ✓ Users ask follow-up questions
- ✗ Single-turn Q&A APIs no conversation
- ✗ Memory grows large adds token cost

PATTERN 14

RAG Fusion

Problem: Any single query only retrieves one perspective of the answer different phrasings find different chunks.

```
Original Query: "best chunking strategy"
  │
  ▼
[LLM Generate Variants]
  ├── "optimal text splitting methods"
  └── "chunk size recommendations"
  │
  ▼
[Run ALL queries in parallel]
  │
  ▼
[Reciprocal Rank Fusion (RRF)]
Combines ranked lists
```

HOW IT WORKS

- One original query becomes 4-5 variants
- All variants retrieve independently
- RRF merges ranked result lists
- Documents appearing across multiple queries score higher

- ✓ Complex questions with many angles
- ✓ Recall matters more than latency
- ✗ Simple factual lookups overkill
- ✗ Strict latency budget parallel calls


AI Coach John
PROITBRIDGE Strive For Better Future
Follow
Repost

DECISION TREE

Choosing the Right Pattern

```
START: What is the core problem?
─────────────────────────────────

Query is vague / too short?
├── Very short (1-3 words) → HyDE
└── Poorly worded → Query Rewriting

Retrieved docs are bad?
├── Sometimes irrelevant → CRAG
├── Docs contradict each other → Self-RAG
└── Need multiple angles → RAG Fusion

Answer needs multiple steps?
├── 2-3 retrieval hops → Agentic RAG
├── Real-time info needed → CRAG (web fallback)
└── Long-form generation → FLARE

Performance / scale issue?
├── Mixed query complexity → Adaptive RAG
├── Large doc sets, need speed → Speculative
└── Swappable components needed → Modular RAG

Special data types?
├── Entities & relationships → GraphRAG
├── Images, charts, tables → Multi-Modal RAG
└── Multi-turn chat → Conversational RAG
```



AI Coach John
PROITBRIDGE
Strive For Better Future
Follow
Repost

COMPARISON

Patterns Comparison Table

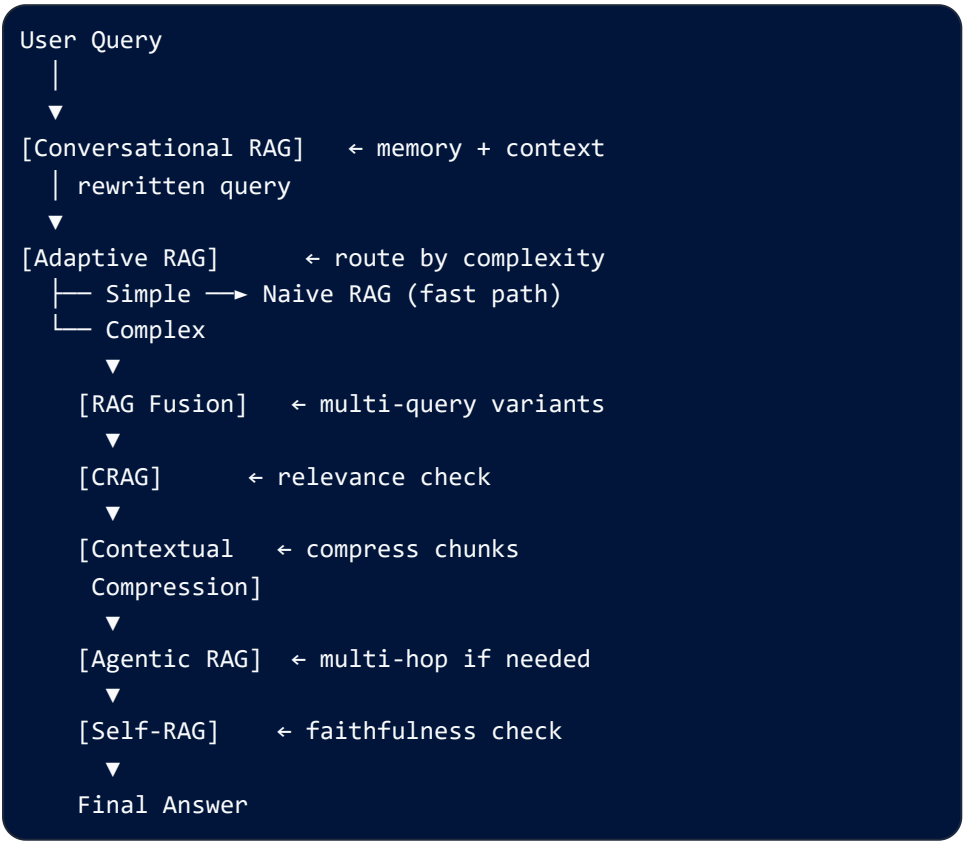
PATTERN	RETR.	LLMS	LATENCY	ACCURACY
Naive RAG	1	1	⚡ fast	⭐⭐
HyDE	1	2	medium	⭐⭐⭐
Query Rewrite	N	2	medium	⭐⭐⭐
CRAG	1+web	2-3	medium	⭐⭐⭐⭐
Self-RAG	1-N	3-5	slow	⭐⭐⭐⭐⭐
Adaptive RAG	1-N	1-N	varies	⭐⭐⭐⭐
FLARE	N(dyn)	N	slow	⭐⭐⭐⭐⭐
Spec. RAG	N	N+1	medium	⭐⭐⭐⭐⭐
Modular RAG	varies	varies	varies	⭐⭐⭐⭐
Agentic RAG	N	N	slow	⭐⭐⭐⭐⭐
GraphRAG	graph	1-2	medium	⭐⭐⭐⭐⭐
Multi-Modal	N	1-2	medium	⭐⭐⭐⭐
Conv. RAG	1-N	1-2	medium	⭐⭐⭐⭐
RAG Fusion	N×var	1-2	medium	⭐⭐⭐⭐⭐


AI Coach John
PROITBRIDGE
Strive For Better Future
Follow
Repost

ARCHITECTURE

Combining Patterns (Stack)

Real production systems layer multiple patterns together. Here's a battle-tested combination:



Layers add quality. Don't add layers you don't need.


AI Coach John

PROITBRIDGE
Strive For Better Future

Follow

Repost

SUMMARY

Summary Cheat Sheet

PATTERN	SOLVES	USE WHEN
Naive RAG	Getting started	Prototype
HyDE	Short query → poor	Short vague queries
Query Rewrite	Ambiguous queries	Casual chat apps
CRAG	Irrelevant docs	Accuracy critical
Self-RAG	Blind generation	High stakes domains
Adaptive RAG	Waste of compute	Mixed workloads
FLARE	Long-form hallucination	Reports & summaries
Speculative RAG	Speed vs accuracy	Latency matters
Modular RAG	Rigid pipelines	Evolving apps
Agentic RAG	Multi-step reasoning	Complex queries
GraphRAG	Missing relationships	Entity-rich domains
Multi-Modal RAG	Text misses images	Visual docs
Conv. RAG	No memory	Chatbots
RAG Fusion	Single query misses	Broad research

Save this. You now know every major RAG pattern! 🚀

Follow for more AI engineering content →



AI Coach John

PROITBRIDGE
Strive For Better Future

Follow

Repost